



UANL

**FACULTAD DE CIENCIAS
FISICO MATEMATICAS**



FCFM

**LABORATORIO DE PROGRAMACION ORIENTADA
A OBJETOS**

PRACTICA #9

Entrada Salida (I O) en Java

2096042

Jimenez Mendieta Angel Gael

7 de octubre de 2026

INTRODUCCION

En cualquier sistema de software o aplicación, la capacidad de conservar la información más allá del tiempo de ejecución es un requisito fundamental. Hasta ahora, en el desarrollo de nuestro motor de juego RPG, los datos como el inventario, los personajes y los registros de batalla existían únicamente en la memoria temporal del sistema mediante estructuras como listas. Esto significaba que, al cerrar el programa, toda la información generada desaparecía por completo. En esta práctica abordamos este problema implementando la persistencia de datos a través del manejo de Entrada y Salida (I/O) en Java. El objetivo principal es lograr que el estado actual del gremio se guarde físicamente en archivos de texto plano dentro del disco duro. De esta manera, podemos asegurar que el progreso, los objetos recolectados y el historial de combates se conserven intactos para poder ser recuperados o actualizados la próxima vez que el sistema inicie, acercándonos mucho más a la arquitectura funcional de un videojuego real.

EXPLICACION DEL TEMA

El manejo de Entrada y Salida (o I/O por sus siglas en inglés) en Java es el mecanismo que permite a un programa comunicarse con fuentes de datos externas para leer o guardar información. Para lograr que nuestros datos persistan, utilizamos diversas clases especializadas del paquete `java.io`. En primer lugar, la clase `File` nos permite representar y administrar la ubicación de los archivos en el sistema, dándonos la capacidad de verificar si existen o crear las carpetas necesarias desde cero.

Para el proceso de "Salida" (escribir datos en la computadora), utilizamos herramientas como `BufferedWriter` y `FileWriter`. Estas clases nos permiten tomar la información que vive en nuestro programa y traducirla a texto para guardarla en archivos `.txt`. Un detalle muy importante en la escritura es el modo de operación: podemos sobrescribir un archivo por completo (lo cual es ideal para actualizar el inventario o los niveles de los personajes) o usar el modo de "añadir" (conocido como *append* usando `true` en el parámetro), el cual es perfecto para una bitácora, ya que simplemente agrega las nuevas líneas de texto al final del documento sin borrar el historial de las batallas pasadas.

Por otro lado, para el proceso de "Entrada" (cargar los datos al juego), implementamos `BufferedReader` y `FileReader`. Estas herramientas leen los documentos de texto línea por línea de manera muy eficiente, permitiendo que el programa analice el texto (por ejemplo, separando las palabras numéricas y de texto mediante comas) y reconstruya los objetos originales para volver a meterlos a la memoria.

Finalmente, al interactuar con el almacenamiento físico, siempre existe el riesgo de que ocurra una falla externa, como intentar leer un documento que no existe. Por ello, todas estas operaciones requieren el manejo de la excepción

IOException, lo que nos obliga a rodear nuestro código con bloques try-catch para interceptar y gestionar cualquier error de archivo sin que el programa colapse.

CODIGO FUENTE

```
1 import java.io.File;
2 import java.io.IOException;
3 import java.util.ArrayList;
4 import java.util.HashMap;
5 import java.util.Map;
6
7 public class Main {
8     public static void main(String[] args) {
9         try {
10             PersistenciaGremio persistencia = new PersistenciaGremio();
11             ArrayList<Personaje> roster = new ArrayList<>();
12             roster.add(new Druida(nombre: nombre: "Sylva", nivel: nivel: 10, puntosVida: puntos_ 300, 100));
13             roster.add(new Arquero(nombre: nombre: "Legolas", nivel: nivel: 6, puntosVida: puntos_ 150, "Arco", 20, 95));
14             roster.add(new Guerrero(nombre: nombre: "Thorin", nivel: nivel: 9, puntosVida: puntos_ 400, "Hacha", 80));
15             persistencia.guardarRoster(roster);
16
17             ArrayList<String> lineasRoster = persistencia.cargarRoster();
18             System.out.println(x: x: "\n=== Roster cargado desde archivo ===");
19             for (String linea : lineasRoster) {
20                 String[] partes = linea.split(regex: regex: ",");
21                 System.out.println("Nombre: " + partes[0] + " | Nivel: " + partes[1] + " | Vida: " + partes[2]);
22             }
23
24             HashMap<String, Integer> inventario = new HashMap<>();
25             inventario.put(key: key: "Poción de vida", value: value: 8);
26             inventario.put(key: key: "Flecha élfica", value: value: 30);
27             inventario.put(key: key: "Pergamino de fuego", value: value: 3);
28             persistencia.guardarInventario(inventario);
29
30             HashMap<String, Integer> inventarioCargado = persistencia.cargarInventario();
31             System.out.println(x: x: "\n=== Inventario cargado desde archivo ===");
32             for (Map.Entry<String, Integer> e : inventarioCargado.entrySet()) {
33                 System.out.println(e.getKey() + " -> " + e.getValue());
34             }
35
36             persistencia.agregarEntradaBitacora(entrada: entrada: "Sylva atacó a Malachar (daño: 240)");
37             persistencia.agregarEntradaBitacora(entrada: entrada: "Legolas sin flechas no pudo atacar");
38             persistencia.agregarEntradaBitacora(entrada: entrada: "Thorin venció a Dragón de Hielo");
39
40             persistencia.agregarEntradaBitacora(entrada: entrada: "Sylva atacó a Malachar (daño: 240)");
41             persistencia.agregarEntradaBitacora(entrada: entrada: "Legolas sin flechas no pudo atacar");
42             persistencia.agregarEntradaBitacora(entrada: entrada: "Thorin venció a Dragón de Hielo");
43             persistencia.mostrarBitacora();
44
45             File carpeta = new File(pathname: pathname: "datos_gremio");
46             System.out.println(x: x: "\n=== Archivos en datos_gremio/ ===");
47             for (File f : carpeta.listFiles()) {
48                 System.out.println(f.getName() + " (" + f.length() + " bytes)");
49             }
50         } catch (IOException e) {
51             System.out.println("Error de archivo: " + e.getMessage());
52         }
53     }
54 }
```

EJECUCION

```
Problems 12  Output  Debug Console  Terminal  Ports

PS C:\Users\EQUIPO\Downloads\poo> & 'C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.29.30133\bin\Hostx64-x64\cmd.exe' /c 'C:\Users\EQUIPO\AppData\Roaming\Code\User\workspace
in'
[IO] Carpeta 'datos_gremio' creada.
[IO] Roster guardado: 3 personajes.
[IO] Roster cargado: 3 registros.

=== Roster cargado desde archivo ===
Nombre: Sylva | Nivel: 10 | Vida: 300
Nombre: Legolas | Nivel: 6 | Vida: 150
Nombre: Thorin | Nivel: 9 | Vida: 400
[IO] Inventario guardado: 3 items.
[IO] Inventario cargado: 3 items.

=== Inventario cargado desde archivo ===
Pergamino de fuego -> 3
Poción de vida -> 8
Flecha élfica -> 30

=== Bitácora de Batallas ===
1. Sylva atacó a Malachar (daño: 240)
2. Legolas sin flechas no pudo atacar
3. Thorin venció a Dragón de Hielo

=== Archivos en datos_gremio/ ===
bitacora.txt (109 bytes)
inventario.txt (60 bytes)
roster.txt (43 bytes)
PS C:\Users\EQUIPO\Downloads\poo>
```

o > Descargas > poo > datos_gremio

ombbre	Fecha de modificación	Tipo	Tamaño
bitacora	07/10/2026 04:51 p. m.	Documento de te...	1 KB
inventario	07/10/2026 04:51 p. m.	Documento de te...	1 KB
roster	07/10/2026 04:51 p. m.	Documento de te...	1 KB

bitacora: Bloc de notas

Archivo Edición Formato Ver Ayuda

Sylva atacó a Malachar (daño: 240)
Legolas sin flechas no pudo atacar
Thorin venció a Dragón de Hielo

roster: Bloc de notas

Archivo Edición Formato Ver Ayuda

Sylva,10,300
Legolas,6,150
Thorin,9,400

inventario: Bloc de notas

Archivo Edición Formato Ver Ayuda

Pergamino de fuego,3
Poción de vida,8
Flecha élfica,30

PREGUNTAS

1. ¿Cuál es la diferencia entre `FileWriter(archivo)` y `FileWriter(archivo, true)`? ¿Qué pasaría con la bitácora si usaras el primero?

El primer formato sobrescribe el archivo por completo cada vez que lo abres (borrando lo que había antes). Al ponerle el `true`, se activa el modo *append*, lo cual significa que el texto nuevo se va a agregar al final del archivo respetando lo que ya existía. Si no hubiéramos usado el modo *append* en la bitácora, cada que registraras una batalla, se borraría el historial y siempre tendrías solo el último turno guardado.

2. ¿Por qué es importante verificar si un archivo existe antes de leerlo? ¿Qué excepción lanzaría Java si intentas leer un archivo que no existe sin verificar?

Es vital verificar con `archivo.exists()` para prevenir que el programa colapse intentando leer información de la nada. Si trataras de leer un archivo inexistente usando `FileReader` directo y sin verificar, Java arrojaría una `FileNotFoundException` (que es hija de la excepción general `IOException`)

3. ¿Qué ventaja tiene `BufferedReader` sobre leer el archivo directamente con `FileReader`? ¿Qué hace el "buffer" en este contexto?

`FileReader` es algo lento porque accede al disco duro leyendo un solo carácter (letra) a la vez. El `BufferedReader`, en cambio, agarra bloques inmensos de texto de un solo jalón y los mete a la memoria temporal (eso es lo que significa "buffer"). Esto hace que comandos como `.readLine()` sean muchísimo más rápidos y amigables.

4. ¿Definir Enums en java, para que sirven investigar y listar en documento las clases similares a `BufferedReader` y `BufferedWriter` y `FileReader` y `FileWriter`?

Un Enum (enumeración) sirve para crear tu propio tipo de dato que solo puede tener un grupo fijo y cerrado de valores (por ejemplo, los días de la semana, o algo como enum `Dificultad { FACIL, MEDIO, DIFICIL }`). Evita que el usuario meta valores inventados.